# Tutorial 05B: Concrete Strength — When Data Is Sparse

AI Agentic Workflow for Knowledge Discovery

CE 488/5588 — Applied Artificial Intelligence

Spring 2026

**Overview - AI Agentic Workflow.** This tutorial series is the *starting point* for a much richer journey. Beyond simply chatting with an AI in your terminal, we will build toward an **AI agentic workflow** where you learn to:

- **Develop AI methods** — design, implement, and evaluate ML pipelines and models.
- **Call AI/ML tools and classic tools** — orchestrate model inference, data processing, visualization, and testing from a single agent session.
- **Plan and task** — decompose complex projects into structured sub-tasks, assign them to specialized agents, and track progress.
- **Validate** — verify agent outputs against specifications, run automated tests, and close the loop on quality.
- **Guardrail with AI safety** — enforce permission boundaries, prevent destructive actions, and ensure human oversight at every critical step.
- **Orchestrate human–AI collaboration** — decide when the agent acts autonomously and when it must defer to you, creating a productive partnership between human judgment and AI capability.

Today we take the next step. You will use OpenCode not just to ask questions, but to *direct a systematic exploration* of real data — watching the agent read files, run commands, and produce output, step by step. This is the transition from "vibe programming" to *agentic workflow*.

*In this tutorial, we will walk through using **OpenCode** to explore the UCI Concrete Compressive Strength dataset — a real engineering dataset with a fundamental challenge: **extreme sparsity**. For each unique mixture design and curing time, we typically have only* one observation. *This makes standard regression approaches unreliable and forces us to think carefully about what the data can (and cannot) tell us.*
*By the end of this tutorial you will be able to:*

1. *Use OpenCode to direct a systematic exploratory data analysis (EDA)*
2. *Discover and quantify the sparsity problem in real data*
3. *Understand why treating time as just another feature ignores physics*
4. *Create a reusable* SKILL *artifact that captures empirical knowledge*

## 1 From Vibe Programming to Agentic Workflow

In previous tutorials, you used Google Colab with the built-in Gemini assistant. That workflow is easy to use and gives a sense of "vibe programming" — type a question, get an answer, move on. The agent handles everything behind the scenes: installing libraries, running code, displaying results. You are not bothered by the low-level mechanics, but you also don't see *how* those mechanics fit into knowledge discovery.

Tutorial 05B marks a deliberate shift. By switching to **OpenCode**, you encounter a **highly productive AI-agentic, human-in-the-loop workflow** where:

**Tool calling is visible and explicit.** You watch the agent read files, run shell commands, and produce output — you see the knowledge discovery process unfold step by step.

**Low-level mechanics are hidden, but now they are *accountable*.** You still don't need to `pip install pandas` yourself, but you are aware that an agent is doing it on your behalf — and that this agent is "in flight" with your exploration. This builds meta-cognitive awareness of what the agent is doing and why.

**You learn to direct, not just ask.** This tutorial teaches you to formulate structured prompts that guide the agent through systematic exploration, rather than vague "tell me about this data" requests. This is the transition from chat-based AI to agentic workflow.

**The SKILL artifact captures empirical knowledge.** After exploring the data, you will create a reusable Markdown skill that codifies what you learned — transforming a one-off exploration into reusable expertise for future sparse longitudinal datasets.

This shift is the core learning objective of Tutorial 05B: you should appreciate that working with an AI coding agent is not just about getting answers faster — it is about developing a *new way of thinking* about how tools, data, and domain knowledge interact in the discovery process.

## 2 The Concrete Dataset

We use the UCI Concrete Compressive Strength dataset [1], available at `UCI-Concrete Data/Concrete_Data_CSV.` This dataset contains laboratory measurements of concrete compressive strength for various mixture designs, tested at different curing times.

**Features (7 components)**

1. Cement ($kg/m^3$)
2. Blast Furnace Slag ($kg/m^3$)
3. Fly Ash ($kg/m^3$)
4. Water ($kg/m^3$)
5. Superplasticizer ($kg/m^3$)
6. Coarse Aggregate ($kg/m^3$)
7. Fine Aggregate ($kg/m^3$)

**Additional columns**

- **Age** (days) — curing time, ranging from 1 to 365 days
- **Strength** (MPa) — measured compressive strength (the target)
- Two derived columns (28-day equivalent strength and NSC flag) that we will ignore for this exploration

**Engineering context** Concrete is the most important material in civil engineering. Its compressive strength is a *highly nonlinear function* of the ingredients and the curing time [1]. A specific mixture will gain strength over time as the cement hydrates — this is a *growth curve*. The question we confront in this tutorial is: **can we learn this growth curve from sparse, real-world data?**

## 3 Why This Data Is Challenging

Before we run any code, let us think about what makes this dataset difficult for machine learning. There are four interrelated challenges.

### 3.1 Challenge 1: Supervised Setting Exists — But With Caveats

The dataset provides features (7 components) and a target (strength), with Age as an input. This creates a supervised regression problem. However, treating Age as just another feature ignores the physics of concrete curing.

**Key insight:** Age is not like Cement or Water. It is ordered, it has a known functional relationship (logarithmic curing), and the same mixture tested at different ages is *not* an independent data point in the statistical sense. There is only one "true" strength curve per mixture.

### 3.2 Challenge 2: Extreme Sparsity

For each unique mixture design and curing time, we typically have only *one observation*. This means:

- Most (mixture, age) combinations appear only once
- Some mixtures have measurements at 8 different ages; most have only 1–3
- Standard regression may overfit to unique observations

We will quantify this sparsity empirically in Section 4.3.

### 3.3 Challenge 3: Severe Age Imbalance

Most data is concentrated at 28 days (the standard curing time for concrete testing). This means:

- Models may learn general 28-day patterns rather than true curing behavior
- Predictions at other ages will be uncertain
- Standard train/test splits may not reveal this problem

We will visualize the age distribution in Section 4.2.

### 3.4 Challenge 4: Zero-Inflated Components

Some components (Blast Furnace Slag, Fly Ash, Superplasticizer) are zero in a large fraction of samples. This creates a *mixture of subpopulations*:

- Some mixtures use only cement and aggregates
- Others use supplementary cementitious materials (slag, fly ash)
- These subpopulations may follow different strength-development curves

## 4 Exploring with OpenCode

Now we put theory into practice. You will use OpenCode to direct a systematic exploration of the concrete dataset. The goal is not to get answers quickly — it is to *learn how to direct an agent through a structured investigation.*

**Prerequisite:** You should have completed Tutorial 05A and have OpenCode installed and connected to a model. If not, go back and complete Tutorial 05A first.

**How to use this section:** For each experiment, you will:

1. Read the *Objective* to understand what you are looking for
2. Type the provided OpenCode prompt in the message box (`i` to start typing)
3. Press `Ctrl+S` to send the prompt
4. Review the agent's output and compare it with the *Expected Results*
5. Read the *Discussion* to reflect on what the results mean

### 4.1 Experiment 1: Basic Dataset Overview

**Objective:** Load the dataset and understand its basic structure.

**OpenCode prompt:**

```
Load the CSV file at UCI-Concrete Data/Concrete_Data_CSV.csv. Report:
- Total rows and columns
- Column names with descriptions
- Data types for each column
- Any obvious missing values
```

**Expected results:**

- 1,030 rows, 11 columns
- 7 component features, Age, Strength, and 2 derived columns
- All numeric types (float64 or int64)
- No missing values (after dropping one fully-missing row)

**Discussion:** Notice that the agent had to handle a *BOM (Byte Order Mark)* character at the start of the file. This is a common issue with CSV files from different sources. The agent's ability to detect and handle this automatically is part of why OpenCode is more powerful than a simple chat interface — it *sees* the file as it actually exists.

### 4.2 Experiment 2: Age Distribution

**Objective:** Understand how the curing times are distributed across the dataset.

**OpenCode prompt:**

```
Analyze the distribution of the Age column:
- How many unique values?
- Count and percentage for each unique age
- Show a visual bar representation
- Identify the dominant age(s)
```

**Expected results:**

- 14 unique ages: 1, 3, 7, 14, 28, 56, 90, 91, 100, 120, 180, 270, 360, 365 days
- 41.3% of data is at 28 days
- Only 0.2% at 1 day, 1.3% at 270 days
- Heavy right skew: most data concentrated at early-to-standard curing times

| | | |
|---|---|---|
| 1 | 0.2% | 2 |
| 3 | 13.0% | 134 |
| 7 | 12.2% | 126 |
| 14 | 6.0% | 62 |
| 28 | 41.3% | 425 |
| 56 | 8.8% | 91 |
| 90 | 5.2% | 54 |
| 91 | 2.1% | 22 |
| 100 | 5.0% | 52 |
| 120 | 0.3% | 3 |

**Figure 1: Age distribution in the concrete dataset**

**Discussion:** The 28-day concentration is not surprising — it is the standard curing time for concrete testing. But it means that any model trained on this data will be heavily influenced by 28-day patterns. Predictions at other ages will be uncertain, especially for long-term curing (180+ days), where we have only 3.2% of the data.

### 4.3 Experiment 3: Sparsity Analysis

**Objective:** Quantify how sparse the data is by counting unique (mixture, age) combinations.

**OpenCode prompt:**

```
Treat the 7 mixture components as a single identifier.
Report:
- How many unique mixture designs exist?
- Average rows per unique mixture
- Maximum rows per mixture
- What fraction of mixtures have only 1 observation?
```

**Expected results:**

- 427 unique mixture designs (out of 1,030 rows)
- Average 2.41 rows per mixture
- Maximum 20 rows per mixture
- 57.6% of mixtures have only 1 age measurement

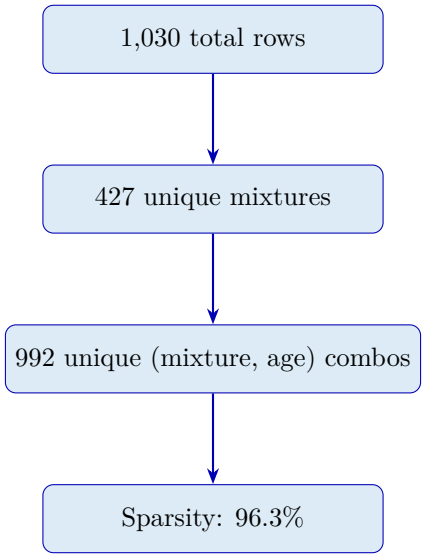


**Figure 2: Sparsity hierarchy in the concrete dataset**

**Discussion:** The **sparsity ratio** is 96.3% — meaning that 96.3% of (mixture, age) combinations appear only *once*. This is extreme. For comparison, a well-designed experiment might have 5–10 replicates per condition. Here, most conditions have exactly 1 observation.

This sparsity has two major implications:

1. **Overfitting risk:** A model trained on mostly unique observations may memorize the training data rather than learning general patterns.
2. **Generalization uncertainty:** When predicting strength for a new mixture at a new age, the model has no direct evidence to rely on.

### 4.4 Experiment 4: Growth Curve Discovery

**Objective:** Find mixtures that have measurements at multiple ages and examine how strength changes with time.

**OpenCode prompt:**

```
Find mixtures that have measurements at multiple ages.
For each such mixture:
- List the ages measured
- List the corresponding strengths
- Show how strength changes with age
Report:
- How many mixtures have multi-age data?
- What fraction of total rows are growth curve data?
```

**Expected results:**

- 181 mixtures have multi-age measurements (42.4% of mixtures)
- 784 rows are growth curve data (76.1% of total)
- Top mixtures have up to 8 age measurements
- Strength generally increases with age (as expected)

**Example: Top mixture by age count**
Ages: 1, 3, 7, 14, 28, 90, 180, 270 days
Strengths: 12.6, 26.1, 33.2, 36.9, 44.1, 47.2, 51.0, 55.2 MPa

**Figure 3: Growth curve example — strength vs. age for one mixture**

**Discussion:** The growth curve data is valuable: it shows that strength increases with age, but the rate of increase slows down (logarithmic behavior). However, even the richest mixtures have only 8 data points across 270 days. This is far too few to fit a reliable curing model for each mixture individually.

The key tension emerges: **we know that strength evolves with age according to physics, but we have too few observations per mixture to learn that evolution empirically.** This is the fundamental challenge of sparse longitudinal data.

### 4.5 Experiment 5: Feature–Target Relationships

**Objective:** Understand how each feature correlates with strength.

**OpenCode prompt:**

```
Compute the correlation between each feature and the target strength.
Report:
- Strongest positive predictor
- Strongest negative predictor
- Any surprising correlations
```

**Expected results:**

- Strongest positive: Cement (+0.50)
- Strongest negative: Water (-0.29)
- Superplasticizer: +0.37 (moderate positive)
- Age: +0.33 (moderate positive)

| Feature | Corr | Bar | Interpretation |
|---|---|---|---|
| Cement | +0.50 |  | Strongest positive |
| Superplasticizer | +0.37 |  | Moderate positive |
| Age | +0.33 |  | Moderate positive |
| Blast Furnace Slag | +0.14 |  | Weak positive |
| Fly Ash | -0.11 | | Weak negative |
| Coarse Aggregate | -0.17 | | Weak negative |
| Fine Aggregate | -0.17 | | Weak negative |
| Water | -0.29 | | Strongest negative |

**Figure 4: Correlation between features and strength**

**Discussion:** Cement is the strongest predictor, which makes engineering sense: more cement generally means stronger concrete. Water has a negative correlation, which also makes sense: excess water weakens concrete. These correlations are consistent with domain knowledge.

However, correlations do not tell us about the *time-dependent* nature of strength. Age has a moderate positive correlation (+0.33), but this is confounded by the fact that most data is at 28 days. We need to look at strength *within each mixture* over time, not just across all data.

### 4.6 Experiment 6: Outlier Detection

**Objective:** Identify outliers in strength using the IQR method.

**OpenCode prompt:**

```
Use the IQR method to detect outliers in the target variable.
Report:
- Number of outliers
- Details of outlier observations
- Strength distribution summary (min, max, mean, median, 70th percentile)
```

**Expected results:**

- 4 outliers detected (strength $>$ 79.77 MPa)
- Strength range: 2.33 – 82.60 MPa
- Mean: 35.82 MPa, Median: 34.45 MPa
- 70th percentile: 43.29 MPa (70% of data below this)

**Discussion:** The strength distribution is right-skewed: the mean (35.82) is higher than the median (34.45), and the 70th percentile is only 43.29 MPa. This means most concrete in the dataset has moderate strength (10–50 MPa), with a few high-strength examples. Outliers may represent high-performance concrete mixes, which could be an interesting subpopulation to explore further.

### 4.7 Experiment 7: Data Imbalance Summary

**Objective:** Group observations into time bins and understand the temporal distribution.

**OpenCode prompt:**

```
Group observations into time bins:
- Early (1--7 days)
- Standard (8--28 days)
- Medium (29--90 days)
- Long (91--180 days)
- Very Long (181+ days)
Report the count and percentage in each bin.
```

**Expected results:**

| Bin | Count | % |
|---|---|---|
| Early (1–7 days) | 262 | 25.4% |
| Standard (8–28 days) | 487 | 47.3% |
| Medium (29–90 days) | 145 | 14.1% |
| Long (91–180 days) | 103 | 10.0% |
| Very Long (181+ days) | 33 | 3.2% |

**Discussion:** Nearly half the data (47.3%) is in the 8–28 day range. Only 3.2% is for very long curing times (181+ days). This imbalance means:

- Models will be well-calibrated for short-to-medium curing times
- Predictions for long-term strength will have high uncertainty
- Standard evaluation metrics (e.g., MSE on the full dataset) will be dominated by performance at 28 days

## 5 What We Learned

Through these seven experiments, we have discovered four key facts about the concrete dataset:

1. **Supervised setting exists but is challenging.** The data provides features and a target, but the relationship is highly nonlinear and time-dependent.
2. **Sparsity is extreme.** 96.3% of (mixture, age) combinations appear only once. Most mixtures have only 1–3 age measurements.
3. **Age imbalance is severe.** 47.3% of data is at 28 days; only 3.2% is for long-term curing (181+ days).
4. **Physics matters.** Strength evolves with age according to curing kinetics, not as a simple feature–target mapping. Treating Age as just another feature ignores this physics.

These findings have important implications for modeling:

**Naive approach (treat Age as feature):** Train a standard regression model with all 8 features (7 components + Age). This will work reasonably well for 28-day predictions but will fail for other ages because the model has no mechanism to capture the time-dependent curing process.

**Better approach (physics-informed):** Use domain knowledge to inform the model. For example, use log(Age) instead of Age, or fit separate models for different subpopulations (e.g., mixtures with vs. without slag). This requires more engineering effort but will generalize better.

**Best approach (hierarchical / mixed-effects):** Treat each mixture as a random effect with a shared curing curve. This leverages the growth curve data (76% of rows) while accounting for the sparsity per mixture. This is the most statistically sound approach but requires careful implementation.

In future tutorials, we will explore these approaches in more detail. For now, the key takeaway is: **understanding the data is the first and most important step in any ML project.** Without

this understanding, you risk building models that are statistically convenient but scientifically meaningless.

## 6 Creating Your SKILL

Now that you have explored the data, it is time to create a **SKILL** artifact. A SKILL is a reusable piece of knowledge that captures what you have learned and can be applied to future similar problems.

**What is a SKILL?** A SKILL is a Markdown file that documents:

- **Purpose:** When to use this skill (what kind of data/problems it applies to)
- **Key observations:** The main challenges or patterns you discovered
- **Exploration checklist:** Step-by-step prompts you can use to explore similar data
- **Discussion questions:** Questions to help you think critically about the results
- **What to try next:** Suggestions for modeling approaches

**Your task:** Create a Markdown file called `concrete-sparsity-skill.md` in the tutorial directory. Use the following structure:

```
# Concrete Sparsity Exploration Skill

## Purpose
When to use this skill (sparse longitudinal data with physical time dimension)

## Key Observations
The 4 challenges: supervised setting exists, time != feature, sparse per mixture, ...
    age imbalance

## Exploration Checklist
Step-by-step OpenCode prompts for similar datasets

## Discussion Questions
Prompts for class discussion

## What to Try Next
Suggest modeling approaches without implementing

## References
Link to dataset readme and original papers
```

**Guidelines:**

- Use the findings from your exploration (sparsity ratio, age distribution, etc.)
- Make the exploration checklist *general* — it should apply to any sparse longitudinal dataset, not just concrete
- Include the OpenCode prompts you used, but generalize them for other data
- Do *not* include code implementation (this is a discussion-only tutorial)

**This SKILL is your empirical knowledge.** It captures what you learned through trial-and-error exploration. When you encounter a similar dataset in the future, you can use this SKILL to guide your exploration efficiently. This is the transition from one-off learning to *reusable expertise*.

**Example SKILL prompt for OpenCode:**

```
Create a Markdown file called concrete-sparsity-skill.md with the following ...
    structure:
- Purpose: When to use this skill
- Key Observations: The 4 challenges
- Exploration Checklist: Step-by-step OpenCode prompts
- Discussion Questions: Prompts for class discussion
- What to Try Next: Suggest modeling approaches
- References: Links to dataset and papers
```

## 7 Looking Ahead

Through this tutorial you have learned to use OpenCode for *systematic data exploration*. You discovered that real-world data is often sparse, imbalanced, and challenging to model. You created a SKILL artifact that captures this empirical knowledge.

**What comes next?**

1. **Modeling sparse longitudinal data.** In future tutorials, we will explore approaches like log-transforming Age, using hierarchical models, and incorporating domain knowledge.
2. **Evaluation strategies.** How do we evaluate models when data is sparse and imbalanced? Standard metrics (MSE, $R^2$) may be misleading.
3. **Tool orchestration.** As you become more comfortable with OpenCode, you will learn to chain multiple tools (view, bash, edit) into coherent pipelines.
4. **Human–AI orchestration.** The most important skill is knowing *when* to let the agent act and *when* to take the wheel yourself. You will develop judgment for delegating, reviewing, and overriding agent decisions.

The SKILL you create in this tutorial is the first step toward building a *personal knowledge base* of empirical insights. Over time, these skills will accumulate and become your most valuable asset as a practitioner of AI in engineering.

## References

[1] I-Cheng Yeh, "Modeling of strength of high performance concrete using artificial neural networks," *Cement and Concrete Research*, Vol. 28, No. 12, pp. 1797–1808 (1998).

[2] opencode-ai, "OpenCode: A powerful AI coding agent. Built for the terminal," *GitHub*, 2025. Available: https://github.com/opencode-ai/opencode

[3] opencode-ai, "OpenCode README — Configuration, Usage, and Keyboard Shortcuts," *GitHub*, 2025. Available: https://github.com/opencode-ai/opencode/blob/main/README.md

[4] OpenCode, "Install script," *opencode.ai*, 2026. Available: https://opencode.ai/install

[5] Microsoft, "Windows Subsystem for Linux Documentation," *Microsoft Learn*, 2026. Available: https://learn.microsoft.com/en-us/windows/wsl/